# Java基础

## 第1题：JDK和JRE有什么区别?

JRE是Java运行环境(Java Runtime Environment)，包含了JVM和Java核心类库。

JDK是Java开发工具包(Java Developer's Kit)，包含了JRE和Java常见的开发工具(javac、java、jconsole、jvisualvm...)。

## 第2题：&和&&有什么区别

&：按位与

- 如果左边两边是数值，将两个操作数的二进制对齐后，1 & 1 为1，其余都为0。
- 如果左右两边是boolean值，true底层为1，false底层为0。只有两边为true，结果才为true。当左边为false时，右边继续计算。

&&：短路与

- 左右两边必须是boolean值，只有两边为true，结果才为true。当左边为false时，右边表达式不计算。

## 第3题：return、continue、break的区别

| | continue | break | return |
|---|---|---|---|
| 用在哪里 | 循环体中 | 循环体或switch-case结构中 | 方法体中 |
| 作用 | 跳过本次循环剩下的语句，直接准备下一次循环 | 结束本层循环或switch-case结构 | 结束当前方法 |

## 第4题：说一下你对==和equals的认识，它们有什么差别?(高频)

1、==可以比较引用数据类型也可以比较基本数据类型：

- 比较基本数据类型，比较的是数值
- 比较引用数据类型，比较的地址值 Student s1 = new Student() ; Student s2 = new Student() ; sout(s1 == s2)

2、equals只能比较引用数据类型，是因为equals是Object类中的一个方法，要使用方法只能通过对象进行使用，而对象就是引用数据类型：

- 1默认情况下比较的也是对象的地址值

```
public boolean equals(Object obj) {
    return (this == obj);
}
```

- 可以通过重写Object类中的equals方法实现比较两个对象的属性

# 第5题：如果两个对象的hashCode值一样，则它们用equals()比较也是为true，是不是？(高频)

不是。

hashCode方法和equals方法是两个独立的方法，本质上没有任何的关系。它们都是Object类中的方法，这两个方法可以被子类进行重写，子类重写可以自定义具体的逻辑。

```
public class HashCodeDemo01 {

    public static void main(String[] args) {

        // 定义两个字符串
        String s1 = "重地" ;
        String s2 = "通话" ;

        // 比较HashCode值是否相同
        System.out.println(s1.hashCode() == s2.hashCode());

        // 调用equals方法比较内容是否相同
        System.out.println(s1.equals(s2));

    }

}
```

```
HashCodeDemo01 ×
C:\develop\Java\jdk1.8.0_172\bin\java.exe ...
true
false
```

# 第6题：int和Integer、BigInteger有什么区别？

| | int | Integer | BigInteger |
|---|---|---|---|
| 类型 | 基本数据类型（Primitive Type） | 包装类型（Wrapper Class），位于 `java.lang` 包 | 不可变的任意精度整数类，位于 `java.math` 包 |
| 范围 | 范围是从 -2,147,483,648 到 2,147,483,647 | 范围是从 -2,147,483,648 到 2,147,483,647 | 理论上无限制，取决于可用内存 |
| 适用 | 适用于大多数整数运算，性能较高，因为它是基本类型，不需要对象开销 | 用于将 `int` 转换为对象。提供了许多有用的静态方法，如 `parseInt` 、 `valueOf` 等。 支持空值（ `null` ） | 用于处理超出 `int` 和 `long` 范围的大整数。 提供了许多数学运算方法，如加法、减法、乘法、除法、幂运算等。 支持高精度计算。 |

> 类似的：double，Double，BigDecimal的区别

# 第7题：String是基本数据类型吗？

String不是基本数据类型，基础类型有8种：byte、boolean、char、short、int、float、long、double。String属于引用数据类型！

# 第8题：对字符串操作都有哪些类并详细的介绍一下它们之间的区别？（高频）

对字符串操作的类有：String , StringBuilder , StringBuffer

1、String是不可变的字符串

```
String a = "abc" ;
a = "bcd" ;

不可变指的是字符串常量"abc"不可变,作为字符串的变量(a)是可以变的。
```

2、StringBuilder , StringBuffer是可变的字符串

StringBuilder , StringBuffer之间的区别：StringBuffer是线程安全的，StringBuilder是线程不安全的。

StringBuffer保证线程安全的原理：就是针对每一个方法都添加了一个synchronized关键字，将其声明为了同步方法。

# 第9题：String str="abc"与 String str = new String("abc")的定义方法一样吗？

不一样。

String str = "abc" ; 是创建一个字符串对象abc放入常量池中。

String str = new String("abc"); 创建了两个对象，"abc"字符串对象在常量池中，new String("")在堆内存中又创建了一个新的对象。

# 第10题：String类isEmpty和isBlank的区别？

- isEmpty用于判断字符串是不是为空，这里的空是指长度为0的字符串，即不包含任何字符的字符串。
- isBlank用于判断字符串是不是为空，这里的空是指字符串长度为0或全部由空白字符（例如空格、\t、\n等）构成的字符串。

```java
public class TestStringEmptyBlank {
    public static void main(String[] args) {
        String str1 ="";
        String str2 ="  \t\n  ";
        System.out.println("字符串1[" + str1 +"]，isEmpty：" + str1.isEmpty() + ", isBlank：" + str1.isBlank());
        System.out.println("字符串2[" + str2 +"]，isEmpty：" + str2.isEmpty() + ", isBlank：" + str2.isBlank());
    }
}

```

## 第11题：String底层实现是怎样的？

- JDK9之前，String底层是char[]，每一个字符占2个字节，采用UTF16编码方式。
- JDK9之后，String底层是byte[]，每一个字符可能占1个字节或2个字节。
  - 如果当前字符串所有字符都是Latin1字符集中的字符（这些字符覆盖了大多数西欧语言，如英语、法语、德语、西班牙语等所需的字母和符号，编码值范围是[0-255]），那么当前字符串每一个字符占1个字节，采用Latin1编码方式。
  - 如果当前字符串中有任意一个字符不是Latin1字符集中的字符，
    - 例如中文、韩文、日文等，那么每一个字符占2个字节，采用UTF16编码方式。
    - 例如：emoji表情字符等，那么每一个字符占4个字节，采用UTF16编码方式。

## 第12题：普通for和增强for的区别

作用与应用场景：

- 普通for循环，可以用于所有需要重复执行某些语句的场景。
- 增强for循环，只能用于遍历数组与Collection集合。

使用区别：

- 如果使用普通for遍历数组，需要指定下标值，可以修改数组的元素。
- 如果使用增强for遍历数组，不需要指定下标，但无法修改数组的元素。
- 另外，增强for循环只是一种语法糖，增强for循环遍历数组时编译器仍然会将对应代码转换为普通for循环；增强for循环遍历Collection集合时编译器会将对应代码转换为Iterator迭代器遍历集合的代码。

## 第13题：方法重载与重写的区别

| 区别 | 方法重载（Overload） | 方法重写（Override） |
| --- | --- | --- |
| **两个方法位置** | 同一个类中 或父子类 | 子类中 |
| **权限修饰符** | 无 | ①private 方法不能被重写<br>②重写方法的权限修饰符要满足>=被重写方法的权限修饰符 |
| **其他修饰符** | 无 | final、static 修饰的方法不能被重写 |
| **返回值类型** | 无 | ①基本数据类型和 void：必须相同<br>②引用数据类型：重写方法的返回值类型要满足<=重写方法的返回值类型 |
| **方法名** | **必须相同** | **必须相同** |
| **形参列表** | **必须不同** | **必须相同** |
| **throws 异常列表** | 无 | ①被重写方法没有 throws 编译时异常类型，重写方法就不能 throws 编译时异常类型<br>②被重写方法 throws 编译时异常类型，重写方法 throws 的编译时异常类型必须<=被重写方法 throws 编译时异常类型<br>③throws 运行时异常类型不做任何检查 |
| **什么时候确定调用哪个方法** | 在编译时期确定调用哪一个方法，由传递给方法的实际参数类型和数量决定 | 在运行时期确定调用哪一个方法，这依赖于对象的实际类型 |

# 第14题：综合说下final的作用？

- 修饰在变量上，叫常量，该常量必须初始化，初始化之后值就不能被修改，而常量一般全都是用大写来命名。
- 修饰在类上，该类不能被继承。
- 修饰在方法上，该方法不能被重写。

# 第15题：抽象类与接口的区别

在面向对象编程中，特别是在Java语言中，抽象类（Abstract Class）和接口（Interface）都是用来提供一种形式的抽象和代码复用的方式。尽管它们有相似之处，但它们在设计意图和实现细节上有显著的区别。以下是抽象类和接口的一些关键区别：

**抽象类**

1. **继承限制**：有单继承限制。
2. **成员**：
    - 抽象类可以有多种类型的成员变量，包括实例变量和静态变量。
    - 抽象类可以有构造方法，这有助于子类的初始化。
    - 抽象类可以包含初始化块（静态代码块和非静态代码块）。
    - 抽象类可以包含抽象方法和普通方法。

**接口**

1. **多实现**：一个类可以实现多个接口
2. 成员：
    - 接口中不允许有构造方法和初始化块（静态代码块和非静态代码块）
    - 接口成员变量默认是 `public` 、 `static` 和 `final` 的，即常量。
    - 接口的接口中的方法默认是 `public` 的。JDK9之后允许有private方法。
    - Java8之前接口中的所有方法默认是抽象的，在Java 8之后可以包含默认方法和静态方法。

**设计意图**

- **抽象类**通常用来定义一组相关类的公共行为和状态，以及一些通用的功能实现。
- **接口**则更多地用来定义类的行为规范，而不关心具体的实现细节。它通常用来表达“这个类能做什么”。

# 第16题：谈谈你对面向对象三大特性的理解

面向对象编程（OOP）的三大核心特性分别是封装、继承和多态。这些特性共同构成了面向对象设计的基础，使代码更具模块性、可重用性和可维护性。下面分别解释这三个特性：

1. **封装（Encapsulation）**： 封装是指将数据（属性）和操作这些数据的方法（行为）捆绑在一起，形成一个整体，也就是“对象”。同时，封装还意味着隐藏对象的内部状态和实现细节，只暴露必要的公共接口供外部访问。这样做的好处是可以保护对象的状态不被外部代码随意更改，从而增加了代码的安全性和稳定性。封装还使得对象的实现可以独立于使用对象的代码，提高了代码的可维护性。

   在Java中，封装主要通过访问控制符（如 `private`、 `protected` 和 `public` ）来实现，`private` 关键字可以限制只有类内部的代码可以访问某些属性和方法，而 `public` 则可以让外部代码访问。此外，还可以通过提供getter和setter方法来间接访问和修改属性，以增加对属性的控制。

2. **继承（Inheritance）**： 继承允许创建一个新类（子类），该类继承了现有类（父类）的属性和方法。子类可以复用父类的代码，同时也可以覆盖或扩展父类的行为。继承支持代码的重用，减少了代码重复，使得程序结构更加清晰和模块化。

   在Java中，继承通过 `extends` 关键字实现。

3. **多态（Polymorphism）**： 多态是指同一种事物在不同情况下有不同的表现形式。多态有两种形式：静态多态（也称为编译时多态，如方法重载）和动态多态（也称为运行时多态，如方法重写）。动态多态是通过继承和接口实现的，允许我们使用父类或接口类型的引用指向子类对象，然后调用方法时，会根据对象的实际类型来决定调用哪个方法的实现。这样可以编写出更加灵活和可扩展的代码。

# 第17题：访问修饰符的作用范围由大到小，及各自的范围是什么？可以 修饰什么？

修饰符的作用范围由大到小:public、protected、缺省、private

| | 本类 | 本包的所有类 | 其他包的子类 | 其他包的非子类 |
|---|---|---|---|---|
| private | √ | × | × | × |
| 缺省（不写） | √ | √ | × | × |
| protected | √ | √ | √ | × |
| public | √ | √ | √ | √ |

这些权限修饰符可以修饰的角色：

| | class（外部类） | 成员变量 | 成员方法 | 成员内部类 |
|---|---|---|---|---|
| private | × | √ | √ | √ |
| 缺省（不写） | √ | √ | √ | √ |
| protected | × | √ | √ | √ |
| public | √ | √ | √ | √ |

# 第18题：throw与throws的区别

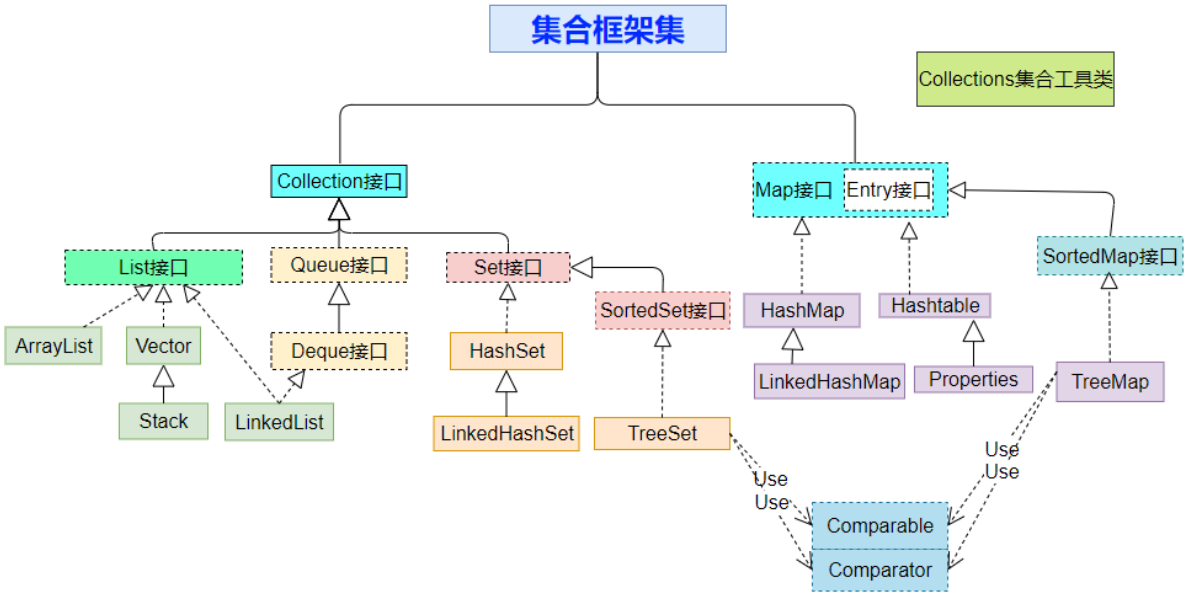
在Java编程语言中，`throw` 和 `throws` 都与异常处理相关，但它们的用途和语法有着明显的区别：

1. **throw**
   - `throw` 是一个关键字，用于在代码中显式地抛出一个异常。
   - 它后面跟着一个异常对象，这个对象通常是某个异常类的实例。
   - `throw` 通常出现在方法体内部，当特定条件满足时执行。
   - 使用 `throw` 抛出异常会立即终止当前方法的执行，并将控制权转移到调用栈上的上一层，寻找适当的异常处理器（如 `catch` 块）。
2. **throws**
   - `throws` 也是一个关键字，但它用于方法的声明部分，而不是方法体内部。
   - 它后面跟着一个或多个异常类的名称，这些异常是方法可能抛出的，用逗号分隔。
   - 当一个方法不能或不应该处理它内部发生的某些异常时，可以使用 `throws` 来声明这些异常，这样调用者必须处理这些异常，要么捕获处理，要么继续声明抛出。
   - `throws` 提供了对调用者的一种合同，表明方法可能抛出哪些类型的异常，使调用者能够做出相应的准备。

总结来说：

- `throw` 用于抛出具体的异常对象，它意味着一个异常正在发生。
- `throws` 用于声明方法可能抛出的异常类型，它意味着如果在方法内部发生这些异常，方法本身不会处理它们，而是将它们抛给调用者。

# 第19题：Java的集合容器都有哪些？(高频)




## 第20题：Collection和Collections有什么区别？

Collection 是一个单列集合的顶层接口。

Collections是集合类的一个工具类，包含了对集合元素进行排序和线程安全等各种操作方法。

## 第21题：如何把一个线程不安全的List集合转成线程安全的List集合？

方案一：直接使用Collections集合工具类的 `synchronizedList()` 方法

方案二：直接选择 `Vector` 或 `CopyOnWriteArrayList` 。

方案三：自定义一个类，让其去实现List接口，重写List接口中的方法，然后在该方法加同步保证线程安全性。

## 第22题：List、Set、Map 之间的区别是什么？(高频)

| | List | Set | Map |
|---|---|---|---|
| 单列or双列 | 单列集合 | 单列集合 | 双列集合，存储键值对 |
| 索引 | 支持 | × | × |
| 重复 | 重复 | × | key不重复，value可重复 |
| 存取一致性 | 一致 | LinkedHashSet一致，其余不一致 | LinkedHashMap一致，其余不一致 |

## 第23题：HashMap 和 Hashtable 有什么区别？(高频)

不同点：

1、HashMap可以把null作为key或value，Hashtable不可以

2、HashMap线程不安全，效率高。Hashtable线程安全，效率低。

## 第24题：如何决定使用 HashMap 还是 TreeMap？

TreeMap可以对元素进行排序，因此如果需要对元素按照某一种规则进行排序，此时可以选择TreeMap。没有特殊要求一般选择HashMap。

## 第25题：ArrayList 和 LinkedList 的区别是什么？

ArrayList底层实现的数据结构是数组，根据下标查询效率极高，时间复杂度是O(1)，非末尾位置增删需要移动元素，添加元素需要扩容！

LinkedList 的底层数据结构是双向链表，根据下标查询效率低，时间复杂度是O(n)，增删元素时，不需要扩容和移动元素，只需要修改前后结点的引用关系！

理论上数组结构的增删慢，链表结构的增删快，但现在内存拷贝技术性能提高之后，ArrayList 的整体性能反而优于 LinkedList，因为 LinkedList 内部需要创建结点对象，这是一个耗时耗力的过程。

附某次测试结果如下：

```
ArrayList头部插入100000个元素耗时：402
LinkedList头部插入100000个元素耗时：6

ArrayList尾部插入100000个元素耗时：6
LinkedList尾部插入100000个元素耗时：5

ArrayList任意位置插入100000个元素耗时：198
LinkedList任意位置插入100000个元素耗时：14393

ArrayList删除头元素100000个耗时：418
LinkedList删除头元素100000个耗时：2

ArrayList删除尾元素100000个耗时：2
LinkedList删除尾元素100000个耗时：2

ArrayList删除任意位置元素100000个耗时：187
```

```
LinkedList删除任意位置元素100000个耗时：3914
```

# 第26题：如何做到数组和List之间的转换？

List对象转换成为数组：可以调用List的**toArray**方法。

数组转换成为List：调用Arrays的**asList**方法，但该方法得到的是一个不可增删元素的List集合。

# 第27题：ArrayList 和 Vector 的区别是什么？

ArrayList和Vector都是List接口的实现类，它们都是动态数组。

| | Vector | ArrayList |
|---|---|---|
| 版本 | JDK1.0（最古老） | JDK1.2（较新） |
| 线程安全 | 安全 | 不安全 |
| 默认初始化容量 | 10，直接创建长度为10的数组 | 0，<br>先初始化为DEFAULTCAPACITY_EMPTY_ELEMENTDATA，<br>然后在首次add元素时才创建长度为10的数组 |
| 扩容 | 2倍 | 1.5倍 |
| | 扩容频率低，空间利用率低 | 扩容频率高，空间利用率更高 |

> 相关的面试题：
>
> 问：什么是Vector？它是List接口的实现类，是古老的动态数组。
>
> 问：ArrayList默认大小是多少，是如何扩容的？见上表
>
> 问：ArrayList是线程安全的吗？不是
>
> 问：ArrayList为什么不是线程安全的？ArrayList 没有内置的同步机制来确保多线程环境下的操作安全。这意味着在多线程环境下使用 ArrayList 时，需要外部手动添加同步机制，如使用 synchronized 关键字或 Collections.synchronizedList 方法。

# 第28题：如何遍历List集合

List遍历有4种方式：

- 普通for循环
- foreach
- Iterator
- ListIterator

```
import org.junit.Test;
```

```java

import java.util.ArrayList;
import java.util.Iterator;
import java.util.ListIterator;

public class TestListIterate {
    @Test
    public void test1(){
        ArrayList<String> list = new ArrayList<>();
        list.add("尚硅谷");
        list.add("atguigu");
        //使用普通for循环遍历
        for (int i = 0; i < list.size(); i++) {
            String s = list.get(i);
            System.out.println(s);
        }
    }
    @Test
    public void test2(){
        ArrayList<String> list = new ArrayList<>();
        list.add("尚硅谷");
        list.add("atguigu");
        //使用增强for循环遍历
        for (String s : list) {
            System.out.println(s);
        }
    }
    @Test
    public void test3(){
        ArrayList<String> list = new ArrayList<>();
        list.add("尚硅谷");
        list.add("atguigu");
        //使用Iterator迭代器遍历
        Iterator<String> iterator = list.iterator();
        while(iterator.hasNext()){
            String s = iterator.next();
            System.out.println(s);
        }
    }

    @Test
    public void test4(){
        ArrayList<String> list = new ArrayList<>();
        list.add("尚硅谷");
        list.add("atguigu");
        //使用ListIterator迭代器遍历
        ListIterator<String> iterator = list.listIterator();
        System.out.println("从前往后遍历：");
        while(iterator.hasNext()){
            String s = iterator.next();
            System.out.println(s);
        }
        System.out.println("从前往后遍历：");
        while(iterator.hasPrevious()){
            String s = iterator.previous();
            System.out.println(s);
```

```
            }
        }
    }

```

## 第29题：哪些集合类是线程安全的?

Vector：比Arraylist多了个同步化机制（线程安全）

Stack ：堆栈类，是Vector的子类，先进后出，项目中用得并不多。

Hashtable：比Hashmap多了个线程安全，项目中一般使用HashMap。

ConcurrentHashMap：是线程安全的，并且比Hashtable效率高。

## 第30题：栈和队列有什么区别?

> 栈：先进后出（FILO：First In Last Out）。队列：先进先出（FIFO：First In First Out）。

拓展：

栈和队列只是逻辑结构，其物理结构可以是数组，也可以是链表。

核心类库中的栈结构有Stack和LinkdeList。Stack就是顺序栈，它是Vector的子类。LinkedList是链式栈。

| 操作 | 方法 |
| --- | --- |
| 入栈 | push(e) |
| 出栈 | pop(e) |
| 查看栈顶元素 | peek() |

`Queue` 除了基本的 `Collection` 操作外， `队列` 还提供其他的插入、提取和检查操作。每个方法都存在两种形式：一种抛出异常（操作失败时），另一种返回一个特殊值（ `null` 或 `false` ，具体取决于操作）。 `Queue` 实现通常不允许插入 元素，尽管某些实现（如 ）并不禁止插入 。即使在允许 null 的实现中，也不应该将null插入到中，因为null也用作 方法的一个特殊返回值，表明队列不包含元素，此时不好区分。

| | ***抛出异常*** | ***返回特殊值*** |
| --- | --- | --- |
| 插入 | add(e) | offer(e) |
| 移除 | remove() | poll() |
| 检查 | element() | peek() |

`Deque`，名称 *deque* 是“double ended queue （双端队列）”的缩写，通常读为“deck”。此接口定义在双端队列两端访问元素的方法。提供插入、移除和检查元素的方法。每种方法都存在两种形式：一种形式在操作失败时抛出异常，另一种形式返回一个特殊值（`null` 或 `false`，具体取决于操作）。Deque接口的实现类有ArrayDeque和LinkedList，它们一个底层是使用数组实现，一个使用双向链表实现。

| | **第一个元素（头部）** | | **最后一个元素（尾部）** | |
|---|---|---|---|---|
| | *抛出异常* | *特殊值* | *抛出异常* | *特殊值* |
| **插入** | addFirst(e) | offerFirst(e) | addLast(e) | offerLast(e) |
| **移除** | removeFirst() | pollFirst() | removeLast() | pollLast() |
| **检查** | getFirst() | peekFirst() | getLast() | peekLast() |

# 第31题：几种Set集合的区别?

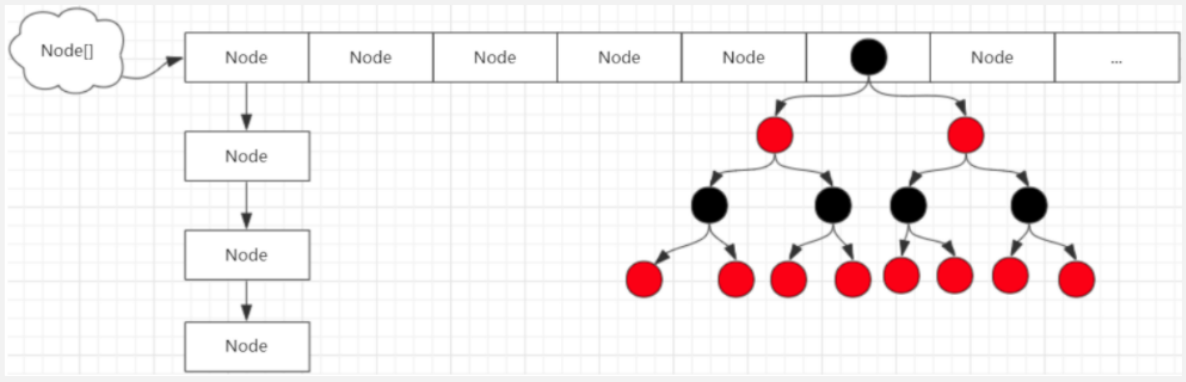
| | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| 底层集合 | HashMap | LinkedHashMap | TreeMap |
| 数据结构 | 数组+单链表+红黑树 | 数组+单链表+红黑树+双链表 | 红黑树 |
| 顺序特点 | 无序 | 插入顺序 | 按元素大小顺序 |
| key支持null | √ | √ | × |
| 线程安全 | 不安全 | 不安全 | 不安全 |

# 第32题：JDK1.7到JDK1.8，HashMap底层发生了什么变化?(高频)

1.8 之后 HashMap 的数据结构发生了变化，从之前的单纯的数组+单链表结构变成数组 + 单链表 + 红黑树。

在HashMap 中存储 K-V 时仅通过 key 来决定每一个 entry 的存储槽位（Node[]中的 index）。多个K-V键值对的index相同，就以链表的形式挂到对应槽位上。

如果只有链表结构，链表长度越长，数据查询的效率越低，链表查询的时间复杂度O(n)。为了提高数据的查询效率，在JDK1.8中就引入了红黑树，`当数组的长度大于64并且某个槽位index下链表长度超过8的时候，此时就会把链表变成红黑树，`红黑树查询的时间复杂度是O(logn)。




# 第33题：HashMap的数组长度有什么特点，扩容原理是什么？（高频）

1、HashMap的默认的初始化的容量：16，也可以手动指定数组长度，数组长度一定是2的n次方，如果指定的长度不是2的n次方，会被纠正为大于该值的最近的2的n次方。这是因为K-V键值对存储到HashMap中时，会通过key 来决定每一个 entry 的存储槽位index，index=（数组的长度-1）& key的hash值，此时数组的长度是2的n次方值，才能确保每一个Node[]数组的位置都有均等的机会。

```
假设数组的长度为10.
hash(key)的二进制值:          xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
table.length-1的二进制值:     00000000 00000000 00000000 00001001
&-----------------------------------------------------------------------------------------
存储位置index的二进制值:      00000000 00000000 00000000 0000x00x

index的可能值: 00000000 00000000 00000000 00000000 => [0]
               00000000 00000000 00000000 00000001 => [1]
               00000000 00000000 00000000 00001000 => [8]
               00000000 00000000 00000000 00001001 => [9]
```

10个位置只有4个位置 可能存储键值对，空间利用率太低，太浪费了

```
假设数组的长度为16.
hash(key)的二进制值:          xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
table.length-1的二进制值:     00000000 00000000 00000000 00001111
&-----------------------------------------------------------------------------------------
存储位置index的二进制值:      00000000 00000000 00000000 0000xxxx

index的可能值: 00000000 00000000 00000000 00000000 => [0]
               00000000 00000000 00000000 00000001 => [1]
               00000000 00000000 00000000 00000010 => [2]
               00000000 00000000 00000000 00000011 => [3]
                        ........
               00000000 00000000 00000000 00001111 => [15]
```

每一个位置都有机会存储键值对，提高空间利用率，从而提高查询效率

2、当HashMap中的元素size值，超过了HashMap的阈值时进行扩容，**每次扩容2倍。阈值的计算 = 最大容量 * 负载因子(0.75)**

第一次扩容： 阈值12 ，扩容完毕以后最大容量：32

第二次扩容： 阈值24 ， 扩容完毕以后最大容量：64

依次类推。

3、还有一个场景也会导致扩容，就是当Node[]数组某一个index位置下的K-V数量达到8，但是数组长度还未达到64时，此时往该index位置继续添加新K-V，就会导致数组扩容。

# 第34题：Lambda表达式是什么？有什么用？

Lambda表达式其实就是匿名函数，它是实现SAM接口（Single Abstract Method，函数式接口）的一种语法糖，它使得Java也成为支持函数式编程的语言。Lambda**写的好**可以极大的减少代码冗余，同时可读性也好过冗长的（啰嗦的）匿名内部类。

Lambda表达式可以用于给函数式接口的变量赋值。例如：当某个方法的参数或返回值类型是函数式接口类型时，Lambda表达式可以被用作参数传递或者返回值。

# 第35题：线程的创建方式？(高频)

JavaSE阶段面试答前两种即可，工作面试答四种。（后两种在JUC部分讲解）

### ① 继承Thread类

```java
public class MyThread extends Thread {

    @Override
    public void run() {
        System.out.println("MyThread...run...");
    }


    public static void main(String[] args) {

        // 创建MyThread对象
        MyThread t1 = new MyThread() ;
        MyThread t2 = new MyThread() ;

        // 调用start方法启动线程
        t1.start();
        t2.start();

    }

}
```

### ② 实现Runnable接口

```java
public class MyRunnable implements Runnable {

    @Override
    public void run() {
        System.out.println("MyRunnable...run...");
    }

    public static void main(String[] args) {

        // 创建MyRunnable对象
        MyRunnable mr = new MyRunnable() ;

        // 创建Thread对象
        Thread t1 = new Thread(mr) ;
        Thread t2 = new Thread(mr) ;

        // 调用start方法启动线程
        t1.start();
        t2.start();

    }

}
```

### ③ 实现Callable接口

```java
public class MyCallable implements Callable<String> {

    @Override
    public String call() throws Exception {
        System.out.println("MyCallable...call...");
        return "OK";
    }

    public static void main(String[] args) throws ExecutionException, InterruptedException {

        // 创建MyCallable对象
        MyCallable mc = new MyCallable() ;

        // 创建F
        FutureTask<String> ft = new FutureTask<String>(mc) ;

        // 创建Thread对象
        Thread t1 = new Thread(ft) ;
        Thread t2 = new Thread(ft) ;

        // 调用start方法启动线程
        t1.start();

        // 调用ft的get方法获取执行结果
        String result = ft.get();

        // 输出
        System.out.println(result);

    }

}
```

### ④ 线程池创建线程

```java
public class MyRunnable implements Runnable{

    @Override
    public void run() {
        System.out.println("MyRunnable...run...");
    }

    public static void main(String[] args) {

        // 创建线程池对象
        ExecutorService threadPool = Executors.newFixedThreadPool(3);
        threadPool.submit(new MyRunnable()) ;

        // 关闭线程池
        threadPool.shutdown();

    }

}
```

# 第36题：线程的run()和start()有什么区别？(高频)

start方法：启动线程，start方法只能调用一次，调用start方法以后，线程的状态就变成RUNNABLE状态(可运行的状态)，当线程获取到了CPU的执行权以后，就开始执行run方法中所封装代码。

run方法：封装要被线程执行的代码，一个线程的run方法执行结束，就表示线程的生命周期结束了。

如果直接调用run方法而非start方法，那么此时线程对象只是一个普通Java对象，程序仍然是单线程执行模式，并未开启多线程执行模式。

# 第37题：如何停止一个正在运行的线程？(高频)

**① 使用退出标志，使线程正常退出。**

```java
public class MyThread extends Thread {

    volatile boolean flag = false ;     // 线程执行的退出标记

    @Override
    public void run() {
        while(!flag) {
            System.out.println("MyThread...run...");
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {

        // 创建MyThread对象
        MyThread t1 = new MyThread() ;
        t1.start();

        // 主线程休眠6秒
        Thread.sleep(6000);

        // 更改标记为true
        t1.flag = true ;

    }
}
```

**② 使用stop方法强行终止**，但是stop方法已过时

```java
public class MyThread extends Thread {

```

```java
    @Override
    public void run() {
        while(true) {
            System.out.println("MyThread...run...");
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {

        // 创建MyThread对象
        MyThread t1 = new MyThread() ;
        t1.start();

        // 主线程休眠2秒
        Thread.sleep(6000);

        // 调用stop方法
        t1.stop();
    }
}
```

# 第38题：单例设计模式（高频）

某个类的对象在整个系统运行期间有且只有1个，这样的类被称为单例类。单例类的通用写法与套路被称为单例设计模式。

通常单例类的写法分为饿汉式与懒汉式。

饿汉式：在类初始化时创建单例对象。

懒汉式：在首次获取这个单例对象时，创建它。

饿汉式写法一：

```java
public enum SingleOne {
    INSTANCE
}
```

饿汉式写法二：

```java
public class SingleTwo {
    public static final SingleTwo INSTANCE = new SingleTwo();
    private SingleTwo(){

    }
}
```

饿汉式写法三:

```java
public class SingleThree {
    private static final SingleThree instance = new SingleThree();
    private SingleThree(){

    }

    public static SingleThree getInstance(){
        return instance;
    }
}

```

懒汉式写法一:

```java
public class SingleFour {
    private static SingleFour instance;//这里不new

    private SingleFour(){
    }

    //当外面调用getInstance()时，说明确实需要这个类的对象，此时再new
    public static synchronized SingleFour getInstance(){
        if(instance == null) {
            instance = new SingleFour();
        }
        return instance;
    }
}

```

```java
public class SingleFour {
    private static SingleFour instance;//这里不new

    private SingleFour(){
    }

    public static SingleFour getInstance(){
        if(instance == null) {
            synchronized (SingleFour.class) {
                if (instance == null) {
                    instance = new SingleFour();
                }
            }
        }
        return instance;
    }
}
```

懒汉式写法二:

```java
public class SingleFive {
    private SingleFive(){
    }

```

```
    private static class Inner{
        static SingleFive instance = new SingleFive();
    }

    //Inner类的加载和初始化，是在调用getInstance方法时，用到Inner类，才初始化，才new了SingleFive的对象
    public static SingleFive getInstance(){
        return Inner.instance;
    }
}

```

## 第39题：Java 中 IO 流分为几种?

按功能来分可以分输入流（input）和输出流（output）。

从类型来分可以是字节流和字符流。

把两种方式结合起来就会形成四种流：

1、字节输入流 ----> InputStream，例如：FileInputStream，BufferedInputStream，ObjectInputStream

2、字节输出流 ----> OutputStream，例如：FileOutputStream，BufferedOutputStream，ObjectOutputStream

3、字符输入流 ----> Reader，例如：FileReader，BufferedReader，InputStreamReader

4、字符输出流 ----> Writer，例如：FileWriter，BufferedWriter， OutputStreamWriter

## 第40题：Java序列化是什么?

序列化和反序列化是Java中实现 `对象` 持久化的重要机制。

- 序列化是指将对象的状态信息转换为字节流的过程，以便于将这些字节流保存到文件、数据库或通过网络传输。
- 反序列化则是将字节流恢复为对象的过程。

要使一个对象可序列化，需要实现 Serializable 接口。Serializable 接口是一个标记接口，没有任何方法需要实现。实现 Serializable 接口需要在类中添加一个常量字段：

```
private static final long serialVersionUID = 值;
```

默认类中static、transient修饰的字段不参与序列化。

## 第41题：TCP和UDP协议有什么区别?

| | TCP | UDP |
|---|---|---|
| 是否连接 | 面向连接 | 面向非连接 |
| 传输可靠性 | 可靠的 | 不可靠的 |
| 应用场合 | 传输大量的数据 | 少量数据 |
| 速度 | 慢 | 快 |

TCP（传输控制协议，Transmission Control Protocol）和 UDP（用户数据报协议User Datagram Protocol）是两种常用的传输层协议，它们在功能和特性上有显著的区别。以下是TCP和UDP的主要区别：

1. 连接方式 TCP：面向连接的协议。在数据传输之前，需要先建立连接（三次握手），数据传输完成后需要断开连接（四次挥手）。 UDP：无连接的协议。数据传输前不需要建立连接，直接发送数据报文。
2. 可靠性 TCP：提供可靠的数据传输服务。通过确认应答、重传机制、拥塞控制等手段确保数据的完整性和顺序性。 UDP：不保证数据的可靠传输。数据报文可能丢失、重复或乱序，但不会进行重传或排序。
3. 数据传输方式 TCP：基于字节流的传输方式。数据被分割成多个段（segments），每个段都有序列号，接收端根据序列号重新组装数据。 UDP：基于数据报的传输方式。数据被封装成一个个独立的数据报文（datagrams），每个数据报文独立传输，没有顺序保证。
4. 速度和效率 TCP：由于可靠性高，需要进行多次握手、确认应答等操作，因此传输速度相对较慢。 UDP：由于没有连接建立和确认应答等操作，传输速度快，效率高。
5. 流量控制和拥塞控制 TCP：提供流量控制和拥塞控制机制。通过滑动窗口机制调节发送速率，避免网络拥塞。 UDP：不提供流量控制和拥塞控制机制。发送方可以以最大速率发送数据，可能导致网络拥塞。
6. 报头大小 TCP：报头较大，标准TCP报头至少20字节。 UDP：报头较小，标准UDP报头只有8字节。
7. 适用场景 TCP：适用于需要可靠传输的场景，如文件传输、电子邮件、网页浏览等。 UDP：适用于对实时性要求高、对数据丢失容忍度高的场景，如视频直播、语音通话、在线游戏等。

## 第42题：你都了解哪些AI插件或AI工具，可以帮助程序员高效开发？

> 本题是开放题，答案不限于下面的参考答案，选择其中你知道的回答即可。

以下是几款主流的 **AI 代码生成插件**（支持 JetBrains IDE/VS Code 等），从技术、功能、适用场景等维度进行对比分析，供参考：

| 工具名称 | 开发公司 | 核心技术模型 | 核心优势 | 适用场景 | 定价 |
| --- | --- | --- | --- | --- | --- |
| **GitHub Copilot** | GitHub (微软) | OpenAI GPT-4 Turbo | - 多语言支持广（Python/JS/TS等） - 深度集成 GitHub 生态 - 智能代码补全与测试生成 | 全栈开发、开源项目、企业级 DevOps | $10/月（个人） $19/月（企业） |
| **Amazon CodeWhisperer** | AWS | 自研模型（基于 AWS 代码库） | - **完全免费** - 优化 AWS 服务开发（Lambda/S3等） - 代码安全扫描 | AWS 云原生开发、Python/Java 项目 | **免费** |
| **Codeium** | Codeium Inc. | 自研模型（Cascade） | - **完全免费**（个人版） - 支持 70+ 语言 - 内置代码搜索 & 聊天功能 | 个人开发者、多语言项目、快速原型 | **免费**（企业版收费） |
| **Tabnine** | Tabnine Inc. | 自研模型 + StarCoder | - **企业级隐私**（支持本地部署） - 适应团队代码风格 - 30%+ 代码自动化生成 | 金融/军工等高安全需求开发 | 免费版 + 企业版（定制定价） |
| **通义灵码** | 阿里云 | 通义千问（Qwen3） | - **中文优化** - 支持 Java/微服务/云原生 - 智能体模式（@workspace） | 国内开发者、阿里云生态项目 | **免费**（企业版收费） |
| **CodeArts Snap** | 华为 | 盘古大模型 | - 适配 **昇腾芯片** - 支持硬件/物联网开发 - 企业级安全合规 | 政企/国产化项目、嵌入式开发 | **免费** |
| **Comate** | 百度 | 文心大模型（ERNIE Code） | - **中文语义理解强** - 低代码生成优化 - 适合教育/文档密集型项目 | 国内 Web/AI 开发、教学场景 | **免费** |
| **MarsCode** | 字节跳动 | 豆包1.5-pro + DeepSeek-V3 | - **轻量级 AI 插件** - 适合快速补全与调试 - 字节生态集成（如飞书） | 个人开发者、小程序/H5 开发 | **免费** |
| **Tencent Cloud CodeBuddy** | 腾讯 | 混元 Turbo S | - **微信生态优化**（小程序/H5） - 腾讯云深度集成 - 企业研效看板 | 腾讯云开发者、游戏后端开发 | **免费**（企业版收费） |

以下是几款与 **Trae**（字节跳动的 AI 原生 IDE）同类型的 **AI 开发工具**，它们均采用 **AI 原生设计**，支持自然语言编程、智能代码生成和全流程开发辅助：

| 工具名称 | 开发公司 | AI 模型 | 核心优势 | 适用场景 | 定价 |
| --- | --- | --- | --- | --- | --- |
| **Cursor** | Anysphere | Claude 3.5 Sonnet、GPT-4o、DeepSeek-V3（可自定义） | - **项目级代码理解**（@Codebase） - **Composer 多文件编辑** - **智能预测 & 自动修复** | 全栈开发、复杂项目重构、企业级应用 | **$20/月**（Pro 版） |
| **Windsurf** | Codeium | Cascade（自研）、Claude 3.5、GPT-4o | - **Agent 模式**（多步任务执行） - **上下文固定（Context Pinning）** - **新手友好 UI** | 中小型项目、团队协作、快速迭代开发 | **$15/月**（Pro 版） |
| **Bolt.new** | StackBlitz | 自研模型（未公开） | - **浏览器内全栈开发** - **对话式 AI 编程** - **云端协作** | Web 开发、快速原型验证 | **$20/月**（Pro 版） |
| **Copilot++** | GitHub（微软） | GPT-4 Turbo、Claude 3.5（可选） | - **GitHub 深度集成** - **CI/CD 自动化** - **企业级安全** | GitHub 生态、企业 DevOps 流程 | **$10/月**（个人版） |
| **Continue** | Continue 团队（国际开源社区） | 支持 **Llama 3、DeepSeek、Ollama**（本地部署） | - **完全离线运行** - **隐私优先** - **可微调本地模型** | 金融/军工等高安全需求开发 | **免费**（需本地算力） |
| **Trae** | 字节跳动 | 豆包1.5-pro、DeepSeek R1/V3、GPT-4o（国际版） | - **Builder 模式**（自然语言生成项目） - **中文优化** - **腾讯/阿里云集成** | 国内开发者、全栈 AI 开发 | **免费**（国内版） |
| **CodeFlying** | 码上飞团队 | 自研模型（低代码优化） | - **低代码 + AI 生成** - **全流程自动化** - **适配微信小程序/H5** | 快速应用开发、初创团队 | **免费试用**（企业版收费） |